Linux I/O Redirection

Understanding Standard Streams:

`stdin(0)` - Standard Input: Default is keyboard
`stderr(2)` - Standard Error: Default is terminal
`stdout(1)` - Standard Output: Default is terminal
`FD(n)` - File Descriptor: Reference to an I/O resource

Basic Redirections

`cmd < file`	Read input from a file instead of keyboard.
`cmd > file`	Send standard output to a file. Overwrites existing file.
`cmd >> file`	Append standard output to a file. Creates file if it doesn't exist.

Error Stream Redirections

`cmd 2> file`	Send error output to a file. Overwrites existing file.
`cmd 2>> file`	Append error output to a file. Creates file if it doesn't exist.
`cmd 2>&1`	Redirect stderr to the same destination as stdout.

Combined Redirections

`cmd > file 2>&1`	Send both stdout and stderr to the same file.
`cmd &> file`	Shorthand to redirect both stdout and stderr to the same file.
`cmd &>> file`	Append both stdout and stderr to the same file.
`cmd 2>&1 > file`	Redirect stderr to terminal, stdout to file. Order matters!

Here Documents and Here Strings

`cmd <<< "string"`	Send a single string to a command as input (here string).

`cmd << EOF`
`text`
`EOF`
	Send multi-line text to a command as input (here document).
	Lines are read until the delimiter (e.g., EOF) is reached.

Pro Tips:

The order of redirections matters! For example,

```
cmd > file 2>&1
```

sends both to file, while

```
cmd 2>&1 > file
```

sends `stderr` to terminal and only stdout to file.

Use `|&` as shorthand for `2>&1 |` to pipe both stdout and stderr.